Azure Key Vault Secret Refresh

Day 07 | AZURE / KEY VAULT + APP CONFIGURATION

Use this guide to investigate a specific failure, apply a cause-specific change and verify recovery. Start with evidence from the affected workload. The examples require adaptation to your environment.

How the failure happens

There are multiple caches: the Key Vault reference can pin a version; the configuration provider can cache a value; application objects and connection pools can retain another copy. Refreshing one layer does not prove every consumer switched.

The Scenario An ASP.NET Core app loads a Key Vault reference once and retains the value.

Why Secret rotation, configuration refresh and database reconnection are different steps.

Check The Reference A version-pinned secret reference continues to fetch that version.

Before you start

An existing ASP.NET Core App Configuration integration; versionless Key Vault references; an identity with required data-plane roles; a safe rotation plan; working network paths.

Diagnostic checks

```
// Inspect provider refresh configuration.
// Check versioned vs versionless references.
// Never log the password.
```

Check The Consumer: Does the app read refreshed configuration, or copy it into a singleton at startup?

Check Connections: Existing connections and pools may retain their own credential state.

Apply the cause specific fix

Fix The Confirmed Cause

```
// Inside options.ConfigureKeyVault(kv => { ... })
kv.SetCredential(new DefaultAzureCredential());
kv.SetSecretRefreshInterval(
    TimeSpan.FromMinutes(5));

// ASP.NET Core registration + pipeline:
builder.Services.AddAzureAppConfiguration();
// After builder.Build():
app.UseAzureAppConfiguration();
```

Provider Configuration: Configure versionless references and an interval appropriate to rotation delay and Key Vault limits.

Trigger Refresh: Request-driven middleware or explicit refresher calls perform checks. An interval is not a background polling loop.

Validate the result

```
// Trigger a refresh-eligible request.
// Verify secret version metadata safely.
// Test a NEW authorized database connection.
```

Expected: The application uses the new credential without exposing it in logs.

Consumer Fix: Reload affected clients/pools safely where required; config refresh alone may not recreate them.

Practice exercise

Rotate a lab secret without exposing it. Trigger a refresh-eligible request, verify version metadata safely, and create a new client connection. Check a singleton consumer separately.

Implementation notes

C# snippets are partial, require Azure.Identity and Azure App Configuration packages, an existing connected provider, correct middleware ordering, appropriate data-plane identity roles and network access. ConfigureRefresh may also be needed for ordinary App Configuration keys; Key Vault refresh interval is separate. IOptionsMonitor or dynamic IConfiguration reads must be wired correctly. ASP.NET request activity or explicit refresh attempts are required. Choose secret-overlap/rotation design that avoids an authentication gap.

Interview question

Does refreshing configuration recreate every database pool? No. Consumers must be designed to use updated values and recycle connections safely where necessary.

Record what changed

- The symptom and affected user operation.
- Evidence supporting the root cause and alternatives ruled out.
- The changed configuration and the version or identity involved.
- Positive and negative tests, plus any remaining risks.

Example status

Examples are educational templates or fragments. They were not deployed against a live cloud account or production cluster. A successful syntax check is not proof of runtime compatibility. Test adapted configurations in an isolated environment first.

Official references

Official documentation 1